

Data Types in Python

A **data type** defines the kind of value a variable can hold and what operations can be performed on that value. Python is **dynamically typed**, meaning you do not need to declare the type explicitly; it is assigned automatically based on the value.

1. Numeric Types

a) int (Integer)

- Whole numbers, positive or negative, without decimals.
- Arbitrary precision (no fixed size like in C++/Java).

```
x = 10
```

```
y = -250
```

```
z = 12345678901234567890 # Python handles big integers
```

```
print(type(x)) # <class 'int'>
```

b) float

- Real numbers with decimal points.
- Can also represent scientific notation.

```
a = 3.14
```

```
b = -2.5
```

```
c = 1.2e3 #  $1.2 \times 10^3 = 1200.0$ 
```

```
print(type(c)) # <class 'float'>
```

c) complex

- Numbers with a real and imaginary part.
- Written as $a + bj$ (where j is the imaginary unit).

```
num1 = 2 + 3j
```

```
num2 = 5 - 2j
```

```
print(num1.real) # 2.0
```

```
print(num1.imag) # 3.0
```

```
print(num1 + num2) # (7+1j)
```

2. Sequence Types

a) str (String)

- Ordered, immutable sequence of characters.
- Supports indexing, slicing, and many built-in methods.

```
s = "Python"
```

```
print(s[0])    # P
```

```
print(s[-1])   # n
```

```
print(s[0:3])  # Pyt
```

```
print(s.upper()) # PYTHON
```

b) list

- Ordered, mutable collection of items.
- Can hold mixed data types.

```
my_list = [1, 2, 3, "hello", 4.5]
```

```
my_list[2] = 10
```

```
print(my_list) # [1, 2, 10, 'hello', 4.5]
```

```
print(my_list[1:4]) # [2, 10, 'hello']
```

c) tuple

- Ordered, immutable collection.
- Useful for fixed data.

```
my_tuple = (1, 2, 3, "python")
```

```
print(my_tuple[0]) # 1
```

```
# my_tuple[1] = 10 # Error (tuples are immutable)
```

3. Set Types

a) set

- Unordered collection of unique items.
- Mutable, but elements must be immutable (hashable).

```
my_set = {1, 2, 3, 3, 4}
```

```
print(my_set) # {1, 2, 3, 4}
```

```
my_set.add(5)
print(my_set) # {1, 2, 3, 4, 5}
```

b) frozenset

- Immutable version of a set.

```
f_set = frozenset([1, 2, 3, 2])
print(f_set) # frozenset({1, 2, 3})
```

4. Mapping Type

dict (Dictionary)

- Collection of key-value pairs.
- Keys must be unique and immutable.
- Values can be of any type.

```
student = {"name": "Atharva", "age": 20, "marks": 95}
print(student["name"]) # Atharva
student["age"] = 21 # update value
print(student) # {'name': 'Atharva', 'age': 21, 'marks': 95}
```

5. Boolean Type

bool

- Represents truth values True and False.
- Internally True = 1, False = 0.

```
x = True
y = False
print(5 > 3) # True
print(10 == 2) # False
```

7. None Type

None

- Represents the absence of a value.
- Commonly used for default function return or initialization.

```
x = None  
  
print(x) # None  
  
print(type(x)) # <class 'NoneType'>
```

Type Conversion

Python supports **type casting** between different types.

```
a = "123"  
  
b = int(a) # String to int  
  
c = float(b) # Int to float  
  
d = str(c) # Float to string  
  
print(type(b), type(c), type(d))
```

Type Checking

```
x = [1, 2, 3]  
  
print(type(x)) # <class 'list'>  
  
print(isinstance(x, list)) # True  
  
print(isinstance(x, (list, tuple))) # True
```